

NATIONAL UNIVERSITY OF SINGAPORE

SCHOOL OF COMPUTING
MIDTERM ASSESSMENT FOR
Semester 1 AY2025/26

CS1101S Programming Methodology

Time Allowed: 1 hour 30 minutes

INSTRUCTIONS TO CANDIDATES

1. This question paper contains 21 questions in 7 sections, and comprises 17 printed pages, including this page.
2. You are also provided with an answer sheet on which to write your answers.
3. Use a pen or pencil to write your Student Number in the designated space on the front page of the answer sheet, and shade the corresponding circle completely in the grid for each digit or letter. **DO NOT WRITE YOUR NAME.**
4. You must submit only the answer sheet and no other documents. Do not tear off any pages from the answer sheet.
5. All questions must be answered in the space provided in the answer sheet; no extra sheets will be accepted as answers.
6. Write legibly with a pen or pencil; do not use red ink. Untidiness will be penalized.
7. For each multiple-choice question, shade the circle corresponding to the one correct answer completely. The number of marks awarded is shown next to each question. There is no penalty for an incorrect answer.
8. The full score of this assessment is **89 marks**.
9. This is a **closed-book** assessment, but you are allowed to bring one double-sided A4 / letter-sized sheet of handwritten or printed notes.
10. Where programs are required, write them in the Python 3 language. A reference of some of the pre-declared functions is given in Appendix A, and Appendix B repeats lecture functions relevant to Section B.
11. Unless otherwise stated, all programs use the Python 3 language.
12. In any question, unless it is specifically allowed, your answer must not use functions given in, or written by you for, other questions.

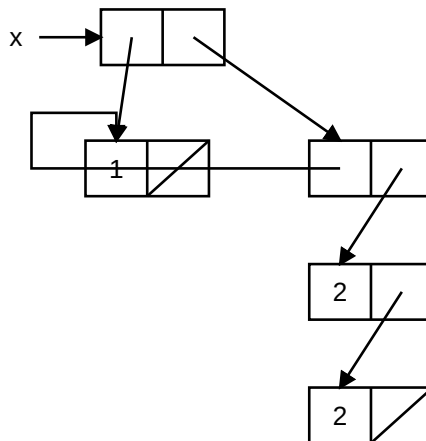
Section A: Box and Pointer Diagrams [8 marks]

1. [4 marks] Draw the box-and-pointer diagram for the value of `x` after the evaluation of the following program. Clearly show where `x` is pointing to.

```
def dup(xs):
    return (None if is_none(xs)
            else pair(head(xs), pair(head(xs), dup(tail(xs)))))

x = dup(llist(llist(1), 2))
```

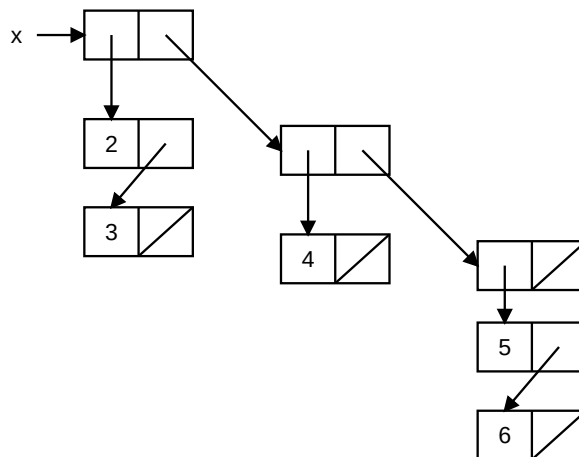
Solution:



2. [4 marks] Draw the box-and-pointer diagram for the value of `x` after the evaluation of the following program. Clearly show where `x` is pointing to.

```
x = map(
    lambda xs: reduce(lambda x, y: pair(x + 1, y), None, xs),
    llist(llist(1, 2), llist(3), llist(4, 5)))
```

Solution:



Section B: Sorting and Orders of Growth [16 marks]

This section uses the functions `merge`, `insert`, `middle`, `take`, and `drop` as presented in the lectures. For each question, choose the order of growth that best characterizes the runtime when applied to descending linked lists of size N .

3. [4 marks]

```
def sort_1(xs):  
    if is_none(xs) or is_none(tail(xs)):  
        return xs  
    else:  
        return merge(llist(head(xs)),  
                      sort_1(tail(xs)))
```

- (a.) $\Theta(1)$
- (b.) $\Theta(\log N)$
- (c.) $\Theta(N)$
- (d.) $\Theta(N \log N)$
- (e.) $\Theta(N^2)$ ✓**
- (f.) $\Theta(N^2 \log N)$
- (g.) $\Theta(N^3)$
- (h.) $\Theta(k^N)$ where k is some constant greater than 1

4. [4 marks]

```
def sort_2(xs):  
    if is_none(xs) or is_none(tail(xs)):  
        return xs  
    else:  
        ys = tail(xs)  
        mid = middle(length(ys))  
        return insert(head(xs),  
                       merge(sort_2(take(ys, mid)),  
                             sort_2(drop(ys, mid))))
```

(a.) $\Theta(1)$

(b.) $\Theta(\log N)$

(c.) $\Theta(N)$

(d.) $\Theta(N \log N)$ ✓

(e.) $\Theta(N^2)$

(f.) $\Theta(N^2 \log N)$

(g.) $\Theta(N^3)$

(h.) $\Theta(k^N)$ where k is some constant greater than 1

5. [4 marks]

```
def sort_3(xs):  
    if is_none(xs) or is_none(tail(xs)):  
        return xs  
    else:  
        size = length(xs)  
        mid = middle(size)  
        return (xs if size <= 1  
                else insert(head(xs), sort_3(tail(xs)))  
                  if size <= 250  
                else merge(sort_3(take(xs, mid)),  
                            sort_3(drop(xs, mid))))
```

(a.) $\Theta(1)$

(b.) $\Theta(\log N)$

(c.) $\Theta(N)$

(d.) $\Theta(N \log N)$ ✓

(e.) $\Theta(N^2)$

(f.) $\Theta(N^2 \log N)$

(g.) $\Theta(N^3)$

(h.) $\Theta(k^N)$ where k is some constant greater than 1

6. [4 marks]

```
def sort_4(xs):  
    if is_none(xs) or is_none(tail(xs)):  
        return xs  
    else:  
        size = length(xs)  
        mid = middle(size)  
        return (xs if size <= 1  
                else merge(sort_4(take(xs, mid)),  
                           sort_4(drop(xs, mid)))  
                if size <= 250  
                else insert(head(xs), sort_4(tail(xs))))
```

(a.) $\Theta(1)$

(b.) $\Theta(\log N)$

(c.) $\Theta(N)$

(d.) $\Theta(N \log N)$

(e.) $\Theta(N^2)$ ✓

(f.) $\Theta(N^2 \log N)$

(g.) $\Theta(N^3)$

(h.) $\Theta(k^N)$ where k is some constant greater than 1

Section C: Higher-order Functions [9 marks]

Natural numbers can be represented with functions using Scott encoding.

```
zero = lambda z: lambda s: z
one  = lambda z: lambda s: s(zero)
two  = lambda z: lambda s: s(one)
three = lambda z: lambda s: s(two)
four  = lambda z: lambda s: s(three)
```

7. [3 marks] Which Python function correctly converts a Scott-encoded number to the corresponding Python number? For example, `scott_to_python(two)` should return 2.

(a.) `scott_to_python = lambda n: n(0)(lambda p: 1 + scott_to_python(p))` ✓

(b.) `scott_to_python = lambda n: n(1)(lambda p: scott_to_python(p) + 1)`

(c.) `scott_to_python = lambda n: n(0)(lambda p: n(0)(p + 1))`

(d.) `scott_to_python = lambda n: n(lambda p: 1 + p)(0)`

(e.) `scott_to_python = lambda n: n(0)(lambda p: 1 + p)`

(f.) `scott_to_python = lambda n: n(1)(lambda p: 1 + p)`

8. [3 marks] Which Python function `add_one` returns the Scott-encoded successor of its Scott-encoded argument? For example, `add_one(two)` should return a function equivalent to three.

(a.) `add_one = lambda n: lambda z: lambda s: s(n)` ✓

(b.) `add_one = lambda n: lambda z: lambda s: n(s)(z)`

(c.) `add_one = lambda n: lambda z: lambda s: s(s(n))`

(d.) `add_one = lambda n: lambda z: lambda s: n(z)(s)`

(e.) `add_one = lambda n: lambda z: lambda s: z(n)`

(f.) `add_one = lambda n: lambda z: lambda s: s(z)`

9. [3 marks] Consider `mystery = lambda n: n(two)(lambda x: x)`. Which Scott-encoded number results from `mystery(four)`?

(a.) zero

(b.) one

(c.) two

(d.) three ✓

(e.) four

(f.) The result is not a Scott-encoded number.

Section D: Higher-order Linked-List Processing [18 marks]

10. [4 marks] What is the result of the following application in linked-list notation?

```
def map_index_helper(f, xs, i):  
    return (None if is_none(xs)  
            else pair(f(head(xs), i),  
                      map_index_helper(f, tail(xs), i + 1)))
```

```
def map_index(f, xs):  
    return map_index_helper(f, xs, 0)
```

```
# map_index(lambda x, i: i * x, llist(4, 5, 6, 7))  
# returns llist(0, 5, 12, 21)
```

```
map_index(lambda x, y: y, build_llist(lambda a: a * 10, 4))
```

- (a.) llist(0, 10, 20, 30)
- (b.) llist(1, 2, 3, 4)
- (c.) llist(0, 1, 2, 3) ✓**
- (d.) llist(10, 20, 30, 40)
- (e.) llist(10, 21, 22, 23)
- (f.) llist(0, 11, 22, 33)

11. [6 marks]

A concrete implementation groups numbers by their remainder modulo 5:

```
def make_nulls(n):
    return None if n == 0 else pair(None, make_nulls(n - 1))

def add_element(i, x, groups):
    return (pair(pair(x, head(groups)), tail(groups))
            if x % 5 == i
            else pair(head(groups),
                      add_element(i + 1, x, tail(groups))))

def helper(xs, groups):
    return (groups if is_none(xs)
            else add_element(0, head(xs), helper(tail(xs), groups)))

def group_by_remainder_5(xs):
    return helper(xs, make_nulls(5))

group_by_remainder_5(llist(29, 30, 31, 32, 33, 34, 35, 36))
# returns llist(llist(30, 35), llist(31, 36), llist(32),
#               llist(33), llist(29, 34))
```

Generalize this implementation. For example, `group(lambda x: x % 5, 6, llist(29, 30, 31, 32, 33, 34, 35, 36))` produces the same five groups followed by None.

Use `map_index` and `build_llist` to implement the abstraction `group(f, n, xs)`, where the result is a linked list of `n` groups and the `i`th group contains the elements `x` with `f(x) == i`, in the same order in which they occur in `xs`. Assume that `f` always returns an integer from 0 to `n - 1`. If you make good use of `map_index` and `build_llist`, no helper function is needed.

You may write up to one helper function.

```
def group(f, n, xs):
    return reduce(
        __[1]__,
        __[2]__,
        xs)
```

Solution:

```
def group(f, n, xs):
    return reduce(
        lambda x, acc: map_index(
            lambda group, i: (pair(x, group)
                              if f(x) == i else group),
            acc),
        build_llist(lambda x: None, n),
        xs)

def group_by_remainder_5(xs):
    return group(lambda x: x % 5, 5, xs)
```

12. [4 marks] Consider the following variant of `reduce` that uses the index of each element. What is the result of the application in linked-list notation?

```
def reduce_index_helper(op, init, xs, i):
    return (init if is_none(xs)
            else op(head(xs), i,
                    reduce_index_helper(op, init, tail(xs), i + 1)))
```

```
def reduce_index(op, init, xs):
    return reduce_index_helper(op, init, xs, 0)
```

```
reduce_index(lambda x, i, y: pair(pair(i, x), y),
             None,
             llist(10, 20, 30))
```

- (a) `llist([2, 10], [1, 20], [0, 30])`
- (b) `llist([30, 2], [20, 1], [10, 0])`
- (c) `llist([0, 10], [1, 20], [2, 30])` ✓
- (d) `llist(llist(2, 10), llist(1, 20), llist(0, 30))`
- (e) `llist(llist(30, 2), llist(20, 1), llist(10, 0))`
- (f) `llist(llist(0, 10), llist(1, 20), llist(2, 30))`

13. [4 marks] Write `map_index(f, xs)` using `reduce_index`. You may write up to one helper function.

```
def map_index(f, xs):
    return reduce_index(
        __[1]__,
        __[2]__,
        xs)
```

Solution:

```
def map_index(f, xs):
    return reduce_index(
        lambda x, i, acc: pair(f(x, i), acc),
        None,
        xs)
```

Section E: Trees [25 marks]

14. [3 marks] Consider the following data structures:

```
data1 = llist("A", "B", "C", llist(None))
data2 = pair(pair("A", pair("B", None)), None)
data3 = llist("A", pair("B", llist("C")), pair(llist("D"), "E"))
```

Which of the above is or are trees of strings?

- (a.) data1 only
 - (b.) data1 and data2 only ✓**
 - (c.) data1 and data3 only
 - (d.) data2 and data3 only
 - (e.) data1, data2, and data3
 - (f.) None of the data structures
15. [5 marks] What sequence of values is printed by print?

```
def map_tree(f, tree):
    return (None if is_none(tree)
            else pair(map_tree(f, head(tree)), map_tree(f,
tail(tree))))

    if is_llist(head(tree))
    else pair(f(head(tree)), map_tree(f, tail(tree))))
```

```
def print_value(x):
    print(x)
    return x
```

```
data4 = llist("F", llist("E", "B", llist("D", "C"), llist("A")))
map_tree(print_value, data4)
```

- (a.) A C D F E B
- (b.) A C D B E F
- (c.) A D C E B F
- (d.) F A E B D C
- (e.) F E B A D C
- (f.) F E B D C A ✓**

16. [3 marks] Using the `map_tree` function from the previous question, how many times is `print` called?

```
data4 = llist("F", llist("E", "B", llist("D", "C"), llist("A")))
def print_value(x):
    print(x)
    return x

map_tree(print_value, map_tree(print_value, data4))
```

- (a.) 0
 - (b.) 5
 - (c.) 6
 - (d.) 12 ✓**
 - (e.) 30
 - (f.) 36
17. [8 marks] Complete `apply_tree(tof, v)`, which takes a tree of unary functions and returns a tree of the same shape with each function applied to `v`. You must not use higher-order linked-list processing functions. You may find the pre-declared functions `is_function` and `pair` useful.

For example:

```
inc = lambda x: x + 1
sqr = lambda x: x * x
tof = llist(inc, llist(sqr, inc))
apply_tree(tof, 2) # returns llist(3, llist(4, 3))
apply_tree(tof, 3) # returns llist(4, llist(9, 4))
```

```
def apply_tree(tof, v):
    return (____[1]____ if is_none(tof)
            else ____[2]____ if ____[3]____
            else ____[4]____)
```

Solution:

```
def apply_tree(tof, v):
    return (None if is_none(tof)
            else pair(head(tof)(v), apply_tree(tail(tof), v))
            if is_function(head(tof))
            else pair(apply_tree(head(tof), v),
                      apply_tree(tail(tof), v)))
```

18. [6 marks] What value is printed by the following program?

```
def compose_tree(tof):  
    return (lambda x: x) if is_none(tof) else (  
        (lambda x: compose_tree(tail(tof))(head(tof)(x)))  
        if is_function(head(tof)) else  
        (lambda x: compose_tree(tail(tof))(compose_tree(head(tof))  
(x))))  
  
add3 = lambda x: x + 3  
dbl  = lambda x: x * 2  
dec5 = lambda x: x - 5  
sqr  = lambda x: x * x  
dec4 = lambda x: x - 4  
  
tof1 = llist(llist(add3, dec4), llist(dbl, llist(dec5, sqr)))  
composition1 = compose_tree(tof1)  
print(composition1(5))
```

- (a.) 0
- (b.) 2
- (c.) 5
- (d.) 9 ✓**
- (e.) 38
- (f.) 39

Section F: Iterative Processes [13 marks]

19. [4 marks] What is the result of evaluating the following Python program in box notation?

```
def m(xs, c):  
    return (c(None) if is_none(xs)  
            else m(tail(xs), lambda x: pair(c(x), head(xs))))  
  
def mystery(xs):  
    return m(xs, lambda x: x)  
  
mystery(list(1, 2, 3, 4))
```

(a.) `[[[None, 1], 2], 3], 4]` ✓

(b.) `[[[None, 4], 3], 2], 1]`

(c.) `[1, [2, [3, [4, None]]]]`

(d.) `[4, [3, [2, [1, None]]]]`

(e.) The program leads to an error.

20. [4 marks] What is the result of evaluating the following Python program?

```
def m(n, c):  
    return n if n <= 1 else m(  
        n - 1, lambda x: c(x + mystery(n - 2)))  
  
def mystery(n):  
    return m(n, lambda x: x)  
  
mystery(6)
```

(a.) `1` ✓

(b.) `8`

(c.) `0`

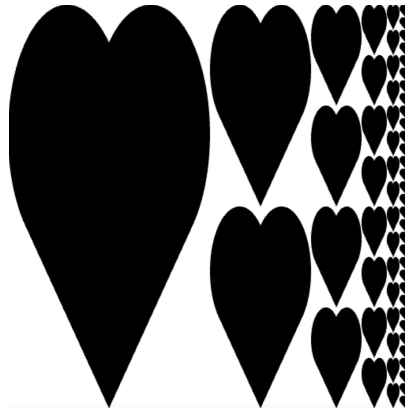
(d.) `13`

(e.) `5`

21. [5 marks]

Convert the recursive fractal process below into an iterative one by completing the three missing expressions in `fractal_helper` and `fractal_iter`.

The primitive `beside` places two runes side by side, and `stack` places one rune above another. For example, `fractal(heart, 6)` produces:



```
def fractal(rune, n):
    if n == 1:
        return rune
    else:
        smaller = fractal(rune, n - 1)
        return beside(rune, stack(smaller, smaller))

def fractal_helper(rune, n, x):
    if n == 1:
        return ____[1]____
    else:
        return ____[2]____

def fractal_iter(rune, n):
    return fractal_helper(rune, n, ____[3]____)
```

Solution:

```
def fractal_helper(rune, n, x):
    if n == 1:
        return x(rune)
    else:
        return fractal_helper(
            rune,
            n - 1,
            lambda r: x(beside(rune, stack(r, r))))

def fractal_iter(rune, n):
    return fractal_helper(rune, n, lambda a: a)
```

APPENDIX A

Python §2 Function Reference

Pairs, linked lists, higher-order functions, type predicates, and mathematical helpers

The following functions are pre-declared in Python §2. **None** denotes the empty linked list; linked-list positions start at zero. The linked-list processing functions below use iterative processes, with the stated time and space bounds. Bounds assume constant-time element operations.

Pairs and linked lists

pair(x, y) Constructs a pair with head **x** and tail **y**. Time and space $O(1)$.

is_pair(x) Returns **True** if **x** is a pair, and **False** otherwise. Time and space $O(1)$.

head(x) Returns the first component of pair **x**. Time and space $O(1)$.

tail(x) Returns the second component of pair **x**. Time and space $O(1)$.

is_none(xs) Returns **True** if **xs** is **None**, and **False** otherwise. Time and space $O(1)$.

is_llist(x) Returns **True** if **x** is a linked list as defined in lectures, and **False** otherwise. Time $O(n)$ and space $O(1)$, where **n** is the length of the chain of tail operations that can be applied to **x**.

llist(x1, ..., xn) Constructs a linked list of **n** elements, in the given order. Time and space $O(n)$; the result contains **n** pairs.

length(xs) Returns the length of linked list **xs**. Time $O(n)$ and space $O(1)$, where **n** is its length.

Search and selection

member(x, xs) Returns the first suffix whose head is identical to **x**, or **None** if none exists. Scalar values match by value; pairs and functions match by object identity. Time $O(n)$ and space $O(1)$.

remove(x, xs) Returns a linked list with the first matching element removed, if present. Matching uses scalar value equality and pair/function identity. Time and space $O(n)$.

remove_all(x, xs) Returns a linked list with every matching element removed, using scalar value equality and pair/function identity. Time and space $O(n)$.

filter(pred, xs) Returns the elements of **xs** for which unary predicate **pred** returns **True**, preserving their order. Time and space $O(n)$.

enum_llist(start, end) Enumerates numbers from **start** in steps of 1 until the next number exceeds **end**. The end is inclusive: **enum_llist(2, 5)** returns **llist(2, 3, 4, 5)**. Time and space $O(n)$ in the result length.

llist_ref(xs, n) Returns the element at position **n**, counting from zero. Time $O(n)$ and space $O(1)$.

Building and processing linked lists

map(f, xs) Returns the result of applying unary function **f** to every element of **xs**, in order. Time and space $O(n)$.

build_llist(f, n) Constructs **n** elements by applying **f** to the numbers 0 through **n** - 1, in order. Time and space $O(n)$.

for_each(f, xs) Applies **f** to each element of **xs** in order, then returns **True**. Time $O(n)$ and space $O(1)$.

reverse(xs) Returns the elements of **xs** in reverse order. Time and space $O(n)$; the process is iterative, but the result itself occupies $O(n)$ space.

append(xs, ys) Returns the elements of **xs** followed by those of **ys**. Time and space $O(n)$ in the length of **xs**. A definition supplied in a question overrides this pre-declared implementation.

reduce(op, initial, xs) Folds from right to left. First **op** is applied to the last element and **initial**, then to the preceding element and that result, and so on. For example:

```
reduce(op, initial, llist(1, 2, 3))  
# equals op(1, op(2, op(3, initial)))
```

Time and space $O(n)$, assuming **op** takes constant time. **reduce** here is the pre-declared linked-list right fold.

Types, output, and mathematics

is_boolean(x), **is_number(x)**, **is_string(x)**, **is_function(x)** Return **True** exactly when **x** has the named type, and **False** otherwise. **is_integer**, **is_float**, and **is_complex** test the corresponding numeric types.

print(*objects) Prints the objects followed by a newline and returns **None**. Where a question needs to print and return a value, it defines **print_value** explicitly.

str(object), **repr(object)** Return informal and debugging string representations, respectively.

llist_to_string(xs) Returns a string representing **xs** in box notation.

math_floor(x) Returns the largest integer less than or equal to **x**.

math_sqrt(x) Returns the square root of **x**.

math_pow(base, exp) Returns **base** raised to the power **exp**.

min(x1, ..., xn), **max(x1, ..., xn)** Return the smallest and largest supplied values, respectively. Time $O(n)$ and space $O(1)$.

APPENDIX B

Sorting Functions Used in Section B

Reference implementations for insert, take, drop, merge, and middle

Assumption. The linked lists supplied to `insert` and `merge` are sorted in ascending order. The split position is `math_floor(n / 2)`.

insert insert one value into a sorted linked list

```
def insert(x, xs):
    if is_none(xs):
        return llist(x)
    elif x < head(xs):
        return pair(x, xs)
    else:
        return pair(head(xs),
                     insert(x, tail(xs)))
```

take keep the first `n` elements

```
def take(xs, n):
    return (None
            if n == 0
            else pair(head(xs),
                      take(tail(xs), n - 1)))
```

drop discard the first `n` elements

```
def drop(xs, n):
    if n == 0:
        return xs
    else:
        return drop(tail(xs), n - 1)
```

merge combine two sorted linked lists

```
def merge(xs, ys):
    if is_none(xs):
        return ys
    elif is_none(ys):
        return xs
    else:
        x = head(xs)
        y = head(ys)
        if x < y:
            return pair(x,
                        merge(tail(xs), ys))
        else:
            return pair(y,
                        merge(xs, tail(ys)))
```

middle find the midpoint used to split a linked list

```
def middle(n):
    return math_floor(n / 2)
```

END OF PAPER